DQC Mini Engine

A small data-quality validation service, written to be read.

Python / FastAPI / Pydantic / SQLAlchemy / SQLite / OpenAI Responses API

A little over 500 lines of Python in six files. A record comes in, the rules check it, the issues are stored in SQLite, and each issue comes back with a severity and a fix someone can act on. I wrote it because data quality is usually the part of a backend that gets bolted on last, and I wanted a small piece of code I could hand over and defend line by line.

What it does

Four endpoints under `/api/v1`, every request and response a Pydantic model, so the OpenAPI output at `/docs` is the documentation instead of a page I forget to update.

- `POST /validate` checks one record, stores what it found and returns the issues.
- `POST /validate/batch` takes up to 100 records, validates them concurrently and returns the counts with the detail.
- `GET /issues` lists stored issues, optionally filtered by severity.
- `POST /suggest` asks a model for a fix when a canned message is not enough.

How I built it

- The contract came first. A record in, a result out, both typed as Pydantic models. That settles the shape of the code before I write any of it.
- Validation is one pure function. `validate_record()` takes a record and returns a list of issues. No session, no request object, no import from FastAPI, so a rule can be exercised with a plain dict and the whole rule set stays in one readable file.
- The rule writes its own fix. Each check returns its suggested_fix next to the failure, for example telling you that a phone number is missing its leading plus. A deterministic rule explains itself better than a language model does, and it never invents a value.
- A broken record does not sink the batch. Every record is wrapped, and an exception becomes a system_error issue on that row only. One bad row returns one failure, not a 500 for all hundred.
- The model sits behind one function. The endpoint calls `_suggestion_provider`, so moving off OpenAI is a one-line change and the rest of the service never learns about it.

What I would change next

- Tests around `validator.py`. It is the cheapest thing to pin down and the only part holding real rules.
- The country check is a 30-code allowlist. It should load the full ISO 3166 table rather than the codes I happened to type.
- The suggestion route calls the synchronous OpenAI client inside an async app. AsyncOpenAI is the honest fix.

Running it

```
pip install -r requirements.txt
python main.py # docs at http://localhost:8000/docs
# OPENAI_API_KEY in .env is only needed for /suggest
```

The code
github.com/namitkharade/dqc-engine

The README covers the layout and both setup paths. The core service runs with no key at all.